SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO* (BL5, BL6)

Assessment Tool Weightage: 10%

QUESTIONS: 20 **TOTAL MARKS: 20**

Annexure A
MOCK INTERVIEW

Code of Conduct:

I Gurusurya G 192311332(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Gurusurya
Signature of the student:

Annexure A

Section 1 — SSL and TLS

Q1. Analyze the weaknesses of SSL and explain why TLS is considered more secure in modern web systems.

SSL (Secure Sockets Layer), especially SSL 2.0 and SSL 3.0, has several well-documented weaknesses:

- Weak and outdated cryptography: SSL relies on algorithms such as RC4, DES, and MD5, all of which are now considered cryptographically broken or too weak against modern computing power.
- POODLE vulnerability: SSL 3.0 is vulnerable to the Padding Oracle On Downgraded Legacy Encryption attack, which lets an attacker decrypt parts of encrypted traffic by exploiting CBC padding.
- Weak handshake integrity: SSL's handshake messages are not fully protected against tampering, making man-in-the-middle downgrade attacks easier.
- No forward secrecy by default: if a server's private key is later compromised, past SSL sessions recorded by an attacker can potentially be decrypted.
- Weak MAC construction: SSL uses a custom MAC scheme rather than the more rigorously analyzed HMAC used in TLS.

TLS (Transport Layer Security) is the successor to SSL and is considered more secure because it removes support for weak ciphers, uses HMAC for message integrity, supports modern authenticated encryption modes (such as AES-GCM), supports Perfect Forward Secrecy through ephemeral Diffie–Hellman key exchange, and has a more rigorously defined and simplified handshake (especially in TLS 1.3, which removes legacy algorithms entirely and reduces the handshake to one round trip). As a result, all modern browsers and servers have deprecated SSL entirely in favor of TLS 1.2 and TLS 1.3.

Q2. Apply the TLS handshake process to explain how secure session keys are established between a client and server.

The TLS handshake establishes a shared secret session key without ever transmitting it in the clear. A simplified TLS 1.2-style handshake proceeds as follows:

- 1. Client Hello: The client sends supported TLS versions, cipher suites, and a random number (client random).
- 2. Server Hello: The server responds with the chosen cipher suite, its own random number (server random), and its digital certificate containing its public key.
- 3. Certificate verification: The client verifies the server's certificate against a trusted Certificate Authority (CA) chain.
- 4. Key exchange: The client generates a pre-master secret and encrypts it with the server's public key (RSA key exchange), or both sides perform an ephemeral Diffie–Hellman exchange (DHE/ECDHE) to jointly derive the pre-master secret, which also provides forward secrecy.
- 5. Session key derivation: Both client and server independently compute the same master secret from the pre-master secret and the two random values, then derive symmetric session keys using a key derivation function (PRF).
- 6. Finished messages: Both sides send a MAC-protected 'Finished' message encrypted with the new session keys to confirm the handshake was not tampered with.
- 7. Secure data exchange: All further communication is encrypted using the fast symmetric session keys (e.g., AES), while the slower asymmetric cryptography is used only during the handshake.

In TLS 1.3, this process is streamlined: the handshake defaults to (EC)DHE key exchange only, completes in a single round trip, and encrypts the server's certificate and remaining handshake messages, reducing exposure and latency.

Q3. Compare SSL and TLS in terms of encryption mechanisms and identify key improvements in TLS.

Aspect	SSL	TLS
Supported ciphers	RC4, DES, 3DES, weak exportable ciphers	AES, ChaCha20; weak ciphers removed
Message integrity	Custom, weaker MAC construction	HMAC (and AEAD integrity in 1.3)

Aspect	SSL	TLS
Key exchange	Mostly static RSA key exchange	Supports ephemeral (EC)DHE for forward secrecy
Handshake	More round trips, less rigorously specified	Streamlined; TLS 1.3 uses 1-RTT (0-RTT resumption)
Known vulnerabilities	POODLE, weak MAC, downgrade attacks	Largely mitigated; legacy modes removed in 1.3
Certificate/handshake protection	Handshake messages largely unencrypted	TLS 1.3 encrypts certificates and most handshake data

Key improvements in TLS: stronger and modern cipher suites, mandatory HMAC-based integrity, support for Perfect Forward Secrecy, a simplified and better-vetted handshake protocol, and (in TLS 1.3) removal of all known-weak legacy algorithms along with encryption of more of the handshake itself.

Q4. Evaluate a scenario where improper certificate validation in TLS leads to a security breach. What could go wrong?

Scenario: A mobile banking app disables certificate validation during development (e.g., by trusting all certificates or ignoring hostname mismatches) and this code ships to production.

- Man-in-the-middle (MITM) attack: An attacker on the same public Wi-Fi network presents a self-signed or fraudulent certificate. Because the app does not validate the certificate chain or hostname, it accepts the attacker's certificate as legitimate.
- Traffic interception: The attacker now sits between the app and the real bank server, decrypting and re-encrypting traffic, capturing login credentials, session tokens, and transaction details.
- Data manipulation: Beyond eavesdropping, the attacker can modify requests in transit, such as changing a transaction's destination account number.
- Root causes typically include: trusting all certificates (empty TrustManager), skipping hostname verification, ignoring expired/revoked certificate checks, or not pinning the certificate for a high-value app.
- Consequences: financial loss, exposed customer data, regulatory penalties, and reputational damage.

Mitigation: enforce full certificate chain validation against trusted CAs, verify hostname matches the certificate's Subject Alternative Name, check revocation status (CRL/OCSP), and use certificate pinning for sensitive mobile applications.

Q5. Analyze the role of digital certificates in SSL/TLS and explain how they ensure authentication.

A digital certificate binds a public key to an identity (such as a domain name or organization), issued and digitally signed by a trusted Certificate Authority (CA). Its role in SSL/TLS authentication includes:

- Identity binding: The certificate contains the server's public key, the domain name(s) it is valid for, the issuing CA, and a validity period.
- Trust chain: The certificate is signed by a CA, which is itself certified by a root CA pre-trusted by the client's operating system or browser, forming a chain of trust.
- Authentication during handshake: When the server presents its certificate, the client verifies the CA's digital signature using the CA's known public key, confirming the certificate has not been tampered with and was genuinely issued to this server.
- Proof of private key possession: The server must also prove it holds the private key corresponding to the certificate (e.g., by signing part of the handshake), preventing an attacker from simply replaying a stolen certificate without the key.
- Revocation checking: Mechanisms like CRLs and OCSP allow clients to confirm a certificate has not been revoked before it expired.

Together, these mechanisms prevent impersonation: a client can be confident it is genuinely talking to the intended server (server authentication), and with mutual TLS, the server can equally authenticate the client.

Section 2 — SET Protocol (Secure Electronic Transaction)

Q6. Apply your understanding of SET protocol to explain how it protects credit card transactions in e-commerce.

SET was developed jointly by Visa and MasterCard to secure card payments over open networks. It protects transactions through several mechanisms:

- Dual signature: SET uses a technique that links order information (sent to the merchant) and payment information (sent to the bank) so that the merchant never sees the customer's card details, and the bank never sees the order details, yet both can be verified as belonging to the same transaction.
- Digital certificates for all parties: Cardholders, merchants, and payment gateways each hold certificates issued by a trusted authority, allowing mutual authentication of every party in the transaction.
- Confidentiality via encryption: Payment information is encrypted so that it is only readable by the payment gateway/issuing bank, not the merchant.
- Integrity: Digital signatures ensure that order and payment data cannot be altered in transit without detection.
- Non-repudiation: Because each party signs their portion of the transaction, none of them can later deny having participated.

In effect, SET separates 'what was ordered' from 'how it was paid for,' ensuring the merchant handles order fulfillment while only the bank ever sees the actual card number.

Q7. Evaluate the limitations of the SET protocol in current online payment systems.

- Complexity: SET required specialized client-side software (an electronic wallet) and certificates for cardholders, merchants, and banks, making it cumbersome to deploy and use.
- Poor user experience: Customers had to install wallet software and manage digital certificates, which was a significant barrier compared to simply typing a card number.
- High infrastructure cost: Merchants and banks needed to deploy and maintain public key infrastructure (PKI) and SET-compliant gateways, which was expensive.
- Slow adoption: Because of this complexity, very few merchants and banks fully adopted SET, and it never achieved the ubiquity its designers intended.
- Superseded by simpler alternatives: Simpler approaches, such as running card transactions over TLS combined with tokenization, 3-D Secure (Verified by Visa/Mastercard SecureCode), and later EMV 3-D Secure, achieved similar security goals with far less friction for users and merchants.

As a result, SET is essentially obsolete today; its ideas (separating payment data from order data, strong mutual authentication) live on conceptually in modern tokenized and 3-D Secure payment flows, but the original protocol is not in active use.

Q8. Compare SET protocol with TLS-based payment security and justify which is more practical today.

Aspect	SET	TLS-based payment security
Scope	End-to-end payment protocol (order + payment)	Secures the communication channel only; payment logic layered on top (tokenization, 3-D Secure)
Client requirement	Special wallet software + personal certificate	Just a standard browser; no special software
Merchant visibility of card data	Merchant never sees card number (dual signature)	Merchant may see card data unless tokenization/PCI-DSS controls are used
Deployment complexity	High — PKI for every participant	Low — TLS is already built into every browser/server
Adoption	Very limited; largely abandoned	Universal — every e-commerce site uses TLS (HTTPS)
Practicality today	Low	High

TLS-based payment security, combined with modern layers like tokenization and 3-D Secure/EMV 3DS, is far more practical today. It reuses infrastructure that is already universal (HTTPS), requires no special client software, and achieves comparable protection of payment data through complementary standards (PCI-DSS, tokenization) rather than a single monolithic protocol. SET's stronger theoretical guarantee of hiding card data from merchants is valuable, but its usability and deployment costs made it commercially unviable.

Section 3 — Malware: Viruses, Worms, Trojans

Q9. Analyze a malware attack scenario and differentiate whether it is caused by a virus, worm, or Trojan.

Scenario: An employee opens an email attachment named 'Invoice.pdf.exe'. Shortly afterward, the file silently installs a backdoor and begins uploading company files to an external server, without spreading further or attaching itself to other files.

Analysis: This is characteristic of a Trojan, not a virus or worm, because:

- It disguises itself as a legitimate file (a fake invoice) rather than exploiting a network vulnerability — a hallmark of Trojan social engineering.
- It does not attach itself to or infect other executable files (which would indicate a virus).
- It does not self-replicate and spread autonomously across the network (which would indicate a worm).
- Its payload (a backdoor for data exfiltration) is a typical Trojan behavior — granting the attacker remote access while appearing benign to the user.

Had the same file instead attached itself to other .exe files and required user execution to spread, it would be classified as a virus. Had it instead scanned the network and copied itself to other machines automatically without user interaction, it would be classified as a worm.

Q10. Apply appropriate mitigation techniques for a worm attack spreading across a corporate network.

- Network segmentation: Isolate infected segments/VLANs immediately to contain the spread and prevent the worm from reaching unaffected subnets.
- Patch the exploited vulnerability: Worms typically spread via unpatched software or protocol vulnerabilities; apply the relevant security patch or update across all hosts.
- Disable vulnerable services/ports: Temporarily block or disable the specific ports/services the worm uses to propagate (e.g., via firewall rules) until systems are patched.
- Deploy IDS/IPS signatures: Update intrusion detection/prevention systems with signatures for the worm's known traffic patterns to detect and block further propagation.
- Endpoint cleanup: Run updated antivirus/EDR tools to detect and remove the worm from infected hosts; reimage severely compromised machines if necessary.
- Monitor and rate-limit traffic: Watch for abnormal outbound connection attempts (a common worm behavior) and apply rate limiting to slow propagation.
- Post-incident review: After containment, conduct a root-cause analysis and reinforce patch management processes to prevent recurrence.

Q11. Evaluate the impact of Trojan attacks on system confidentiality and integrity with an example scenario.

Example scenario: An attacker distributes a Trojan disguised as a 'free PDF converter' tool. Once installed, it opens a backdoor giving the attacker remote access to the victim's machine.

Confidentiality impact: The attacker can access sensitive files, capture keystrokes (via an embedded keylogger), steal saved passwords, and exfiltrate confidential business documents or personal data — a direct breach of confidentiality.

Integrity impact: With remote access, the attacker can modify system files, alter financial records or databases, plant additional malware, or change configuration settings — compromising the integrity of the system and its data.

Broader impact: Because Trojans often establish persistent backdoors, the compromise can remain undetected for a long time, allowing continued data theft and manipulation, and potentially serving as a launch point for further attacks (e.g., ransomware deployment or lateral movement within a network).

This illustrates why Trojans are considered particularly dangerous: unlike a worm's visible network disruption, a Trojan's damage is often silent and long-lasting, undermining both what data can be trusted (integrity) and who can see it (confidentiality).

Q12. Compare viruses, worms, and Trojans based on propagation method and system impact.

Malware Type	Propagation Method	Typical System Impact
Virus	Attaches to a host file/program; needs user action (running the infected file) to activate and spread	Corrupts or modifies files, can degrade performance, may destroy data; damage often tied to infected files
Worm	Self-replicates and spreads autonomously across networks by exploiting vulnerabilities, no user action needed	Consumes network bandwidth and system resources, can carry payloads (e.g., backdoors); spreads very fast
Trojan	Disguised as legitimate/useful software; relies on social engineering to get the user to install it; does not self-replicate	Opens backdoors, steals data, enables remote control; damage is often silent and targeted rather than widespread

Section 4 — Firewalls

Q13. Analyze the working of a packet-filtering firewall and identify its limitations in modern networks.

A packet-filtering firewall examines each individual packet against a set of rules based on header information — typically source/destination IP address, source/destination port, and protocol type — and allows or denies the packet accordingly. It operates at the network layer (and sometimes transport layer) of the OSI model, and each packet is evaluated independently without regard to whether it belongs to an established connection.

Limitations:

- No awareness of connection state: Since each packet is inspected in isolation, the firewall cannot easily distinguish a legitimate response packet from a spoofed or unsolicited one.
- Vulnerable to IP spoofing: Attackers can forge source IP addresses to bypass simple address-based rules.
- No application-layer inspection: It cannot inspect packet payloads, so it is blind to application-layer attacks (e.g., SQL injection, malicious file uploads) hidden inside allowed traffic.
- Complex rule management: As networks grow, the rule sets become large and difficult to maintain, increasing the risk of misconfiguration.
- No user/identity awareness: Rules are based purely on network addresses, not on which user or application is generating the traffic.

Because of these gaps, modern networks typically layer packet filtering with stateful inspection, application-layer (proxy/next-generation) firewalls, and IDS/IPS for deeper protection.

Q14. Apply firewall rules to block unauthorized access while allowing legitimate traffic in a given scenario.

Scenario: A company runs a public web server (port 443/HTTPS) and an internal database server that should only be reachable from the application server, not from the internet.

Example rule set (in order of evaluation):

- Allow inbound TCP traffic to the web server's IP on port 443 (HTTPS) from any source — legitimate customer traffic.
- Allow inbound TCP traffic to the web server's IP on port 22 (SSH) only from the trusted admin IP range — for authorized management access.
- Allow inbound TCP traffic to the database server's IP on port 3306 (or relevant DB port) only from the application server's specific internal IP — enforcing least privilege.
- Deny all inbound traffic to the database server from any other source, including the public internet.
- Deny all inbound traffic on all other ports by default (default-deny policy) — anything not explicitly allowed is blocked.
- Allow outbound traffic for established/related connections (so replies to legitimate requests can return).
- Log and alert on repeated denied connection attempts to detect potential scanning or brute-force activity.

This rule set follows the principle of least privilege and default-deny: only explicitly required traffic is permitted, minimizing the attack surface while keeping the service usable for legitimate users.

Q15. Evaluate the effectiveness of stateful inspection firewalls over stateless firewalls.

A stateless (packet-filtering) firewall evaluates each packet independently, with no memory of previous packets. A stateful inspection firewall maintains a connection state table that tracks the status of active connections (e.g., TCP handshake stage), allowing it to make more intelligent decisions.

Advantages of stateful inspection:

- Context-aware filtering: It can automatically allow return traffic for connections that were legitimately initiated from inside the network, without needing a separate static rule for every possible response.
- Better protection against spoofing: Packets that do not match any known, legitimate connection state (e.g., unsolicited TCP ACK packets) are rejected, mitigating certain spoofing and scanning techniques.
- Improved defense against some DoS patterns: It can detect anomalies like an excessive number of half-open connections (SYN flood indicators).
- Simplified and more secure rule sets: Fewer static rules are needed since the firewall tracks session context dynamically, reducing misconfiguration risk.

Trade-offs:

- Higher resource usage: Maintaining state tables requires more memory and processing than simple stateless filtering.
- Still limited at the application layer: Like stateless firewalls, it generally does not inspect payload content, so it is complemented by application-layer firewalls or IDS/IPS for deep content inspection.

Overall, stateful inspection firewalls are significantly more effective than stateless ones for most modern network environments, offering a good balance of security and performance, which is why they became the de facto standard before next-generation firewalls added even deeper inspection.

Q16. Compare different types of firewalls and recommend the most suitable one for an enterprise network.

Firewall Type	Key Characteristics	Best Suited For
Packet-filtering (stateless)	Filters based on header info only; no connection awareness	Small networks, edge routers, low-cost basic filtering
Stateful inspection	Tracks connection state; smarter than stateless filtering	General-purpose perimeter defense for most organizations
Application-layer / proxy	Inspects payload/content at the application layer; acts as intermediary	Environments needing deep content filtering (e.g., web/email security)
Next-Generation Firewall (NGFW)	Combines stateful inspection with deep packet inspection, intrusion prevention, application awareness, and identity-based rules	Enterprise networks needing comprehensive, layered protection

Recommendation: For a modern enterprise network, a Next-Generation Firewall (NGFW) is the most suitable choice. It combines the connection-awareness of stateful inspection with application-layer visibility, integrated intrusion prevention, malware filtering, and user/identity-based policies — addressing the layered, sophisticated threats enterprises face, which simple packet-filtering or basic stateful firewalls alone cannot handle.

Section 5 — Intrusion Detection and Prevention Systems

Q17. Analyze how an IDS detects malicious activities using signature-based and anomaly-based techniques.

Signature-based detection:

- Compares observed network traffic or system activity against a database of known attack patterns ('signatures'), similar to antivirus definitions.
- Highly accurate and produces low false positives for known, previously catalogued attacks.
- Limitation: cannot detect novel or zero-day attacks that have no matching signature yet, and requires frequent signature database updates.

Anomaly-based detection:

- Builds a baseline/profile of 'normal' network or system behavior (e.g., typical traffic volume, login times, protocol usage) using statistical or machine-learning methods.
- Flags any significant deviation from this baseline as potentially malicious.
- Advantage: capable of detecting previously unseen (zero-day) attacks since it does not rely on known signatures.
- Limitation: prone to higher false-positive rates, since legitimate but unusual behavior (e.g., a new business process) can be misclassified as an attack.

In practice, many IDS deployments use a hybrid approach — signature-based detection for fast, reliable identification of known threats, combined with anomaly-based detection to catch novel attacks that signatures would miss.

Q18. Apply IDS/IPS mechanisms to prevent a distributed denial-of-service (DDoS) attack.

- Traffic baselining and anomaly detection: The IDS/IPS continuously monitors normal traffic volume and patterns, enabling it to quickly recognize the sudden spike in requests characteristic of a DDoS attack.
- Rate limiting: The IPS can automatically throttle the number of requests/connections allowed from a given source or overall, reducing the impact of flooding traffic.
- Signature matching for known DDoS tools: Traffic matching signatures of known DDoS attack tools/botnets can be identified and blocked immediately.
- Automatic blocking (inline IPS action): Unlike a passive IDS, an inline IPS can actively drop malicious packets or reset malicious connections in real time, rather than just alerting.
- Blackholing/sinkholing malicious sources: Traffic from identified attacking IP ranges can be automatically routed to a null route (blackhole) to protect the target.
- Coordination with upstream/cloud DDoS mitigation: For volumetric attacks exceeding local capacity, the IPS can trigger integration with ISP-level or cloud-based scrubbing services.
- Alerting and logging: Security teams receive real-time alerts to investigate and fine-tune mitigation, and logs support post-incident forensic analysis.

The key distinction is that an IPS acts inline and can take these preventive actions automatically and in real time, whereas a pure IDS would only detect and alert, requiring a human or separate system to respond.

Q19. Evaluate the differences between IDS and IPS in terms of response to detected threats.

Aspect	IDS (Intrusion Detection System)	IPS (Intrusion Prevention System)
Deployment mode	Passive — typically monitors a copy of traffic (out-of-band)	Inline — sits directly in the traffic path
Response to threats	Detects and alerts only; requires human or separate system to act	Detects and actively blocks/drops malicious traffic automatically
Impact of false positives	Low risk — an alert is simply reviewed, legitimate traffic unaffected	Higher risk — a false positive can block legitimate traffic in real time
Latency impact	Minimal, since it does not sit inline	Can introduce some latency, since traffic passes through it
Typical use case	Monitoring, forensic analysis, alerting	Real-time prevention of active attacks (e.g., DDoS, exploits)

In evaluation, IDS is valuable for visibility and forensic evidence with minimal risk of disrupting legitimate traffic, while IPS provides active, real-time protection but requires careful tuning to avoid blocking legitimate users. Many enterprises deploy both together: IPS for immediate blocking of well-understood threats, and IDS (or IPS in monitor-only mode) for broader visibility and tuning before enabling full blocking.

Q20. Propose improvements to reduce false positives in an IDS while maintaining high detection accuracy.

- Regular baseline tuning: Continuously update the 'normal' traffic profile used by anomaly-based detection to reflect legitimate changes in business activity, reducing misclassification of new-but-benign patterns.
- Hybrid detection approach: Combine signature-based detection (accurate for known threats) with anomaly-based detection (catches novel threats), cross-validating alerts between the two to filter out noise.
- Machine learning with labeled feedback: Use supervised learning models trained on labeled historical alerts (true positive vs. false positive) to progressively improve classification accuracy over time.

- Contextual/correlation analysis: Correlate alerts across multiple data sources (e.g., firewall logs, endpoint logs, user behavior) using a SIEM, since a single anomalous event is more likely a false positive than a pattern confirmed across multiple systems.
- Whitelisting known-good behavior: Explicitly whitelist trusted applications, IP ranges, and scheduled legitimate high-volume processes (e.g., backups) that would otherwise trigger anomaly alerts.
- Threshold and sensitivity tuning: Adjust detection thresholds based on risk tolerance and asset criticality, rather than using one-size-fits-all sensitivity across the whole network.
- Regular signature/ruleset updates and review: Periodically review and prune outdated or overly broad signatures that are known to generate excessive noise.
- Analyst feedback loop: Establish a process where security analysts label alerts as true/false positives, feeding this back into the tuning process to continuously refine detection logic.

Applying these improvements together allows the IDS to maintain high sensitivity to genuine threats while significantly cutting down the alert fatigue caused by excessive false positives — improving both security effectiveness and analyst efficiency.